SQL-Grounded Vietnamese TableQA

A Multi-Agent Demo for Reliable Answers from Real Tables

Nguyen Huynh Hoang Kha | BSc (Hons) Computer Science | Birmingham City University | 06 June 2026
Supervisors: Nguyen Luu Thuy Ngan and Dang Van Thin | Project artifact: HoangKha1810/ChatBot-OpenViTab

Final System

EM 0.586 | F1 0.674 | Grounding 0.904

Unsupported answers reduced to 4.6%

Problem & Aim

Vietnamese Table Question Answering asks a system to answer natural-language questions from a table, not from free text. The difficult cases in Open-ViTabQA are not only linguistic; they require filtering, comparison, aggregation, row selection, temporal reasoning, and interpretation of abbreviated headers. A direct large language model can write fluent Vietnamese, but it often guesses when the relevant row or column is unclear. The aim of this project is to build a demo-ready pipeline that answers from real data, shows the executable evidence behind each answer, and avoids unsupported responses.

Research Gap

Existing approaches leave a reliability gap. Prompt-only LLMs are flexible but weak at auditability, because the user cannot see which cells produced the answer. Fine-tuning or LoRA improves domain familiarity, yet it does not guarantee row-level grounding. Classic text-to-SQL is more inspectable, but it is sensitive to Vietnamese accents, aliases, unit phrases, merged-style headers, and columns whose meaning is only clear from neighbouring cells. General retrieval-augmented generation retrieves text snippets, but does not enforce table operations such as maximum, minimum, counting, ranking, or numeric comparison. The project therefore combines natural-language understanding, schema-aware SQL generation, deterministic execution, evidence verification, and confidence scoring in one traceable workflow.

Dataset & Demo Data

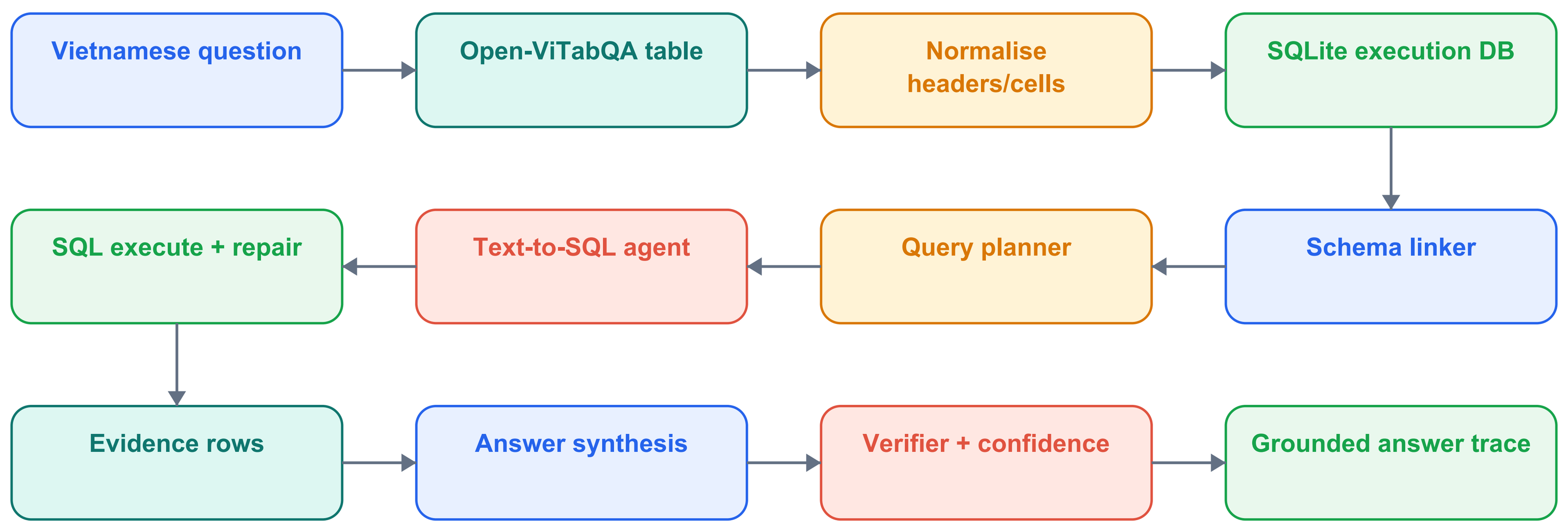
The implementation uses real Open-ViTabQA-style data rather than mock tables. The report split contains 7,928 training questions, 991 validation questions, and 992 test questions. For the live artifact, 329 tables are indexed and converted into SQLite databases so every pipeline run can execute an auditable query. The demo keeps representative domains such as architecture, sports, music, geography, politics, and historical people. A stable filmed case is question 56_3_238: 'Tòa nhà có chiều cao cao nhất có bao nhiêu tầng?' The expected answer is 110, obtained by ordering the real New York building table by the floors column.

Artifact Scope

Data split	7,928 train / 991 validation / 992 test
Demo corpus	329 real Open-ViTabQA tables
Runtime models	bge-m3, qwen2.5-coder:14b, qwen2.5:7b
Hardware target	NVIDIA RTX A5000, 24 GB VRAM
Stable demo case	56_3_238: tallest NYC building has 110 floors

System Architecture

The design treats the table as an executable source of truth. Each answer must pass through schema linking, SQL execution, evidence extraction, answer synthesis, verification, and confidence assignment before it is shown to the user.



Core Method

The coordinator first normalises Vietnamese text, removes formatting noise, creates numeric shadow columns, and stores the table as a SQLite relation named rows. The schema linker embeds the question and column descriptions with bge-m3, then ranks candidate columns. The planner converts the question type into an operation plan: lookup, filter, count, maximum, minimum, comparison, or aggregation. The text-to-SQL agent uses qwen2.5-coder:14b to produce a candidate query, while deterministic guards repair common mistakes such as missing LIMIT, wrong order direction, or invalid column names. The executed SQL returns evidence rows. qwen2.5:7b then writes a concise answer using only that evidence. A verifier checks whether the answer is supported by the rows and whether the predicted value matches deterministic evidence. The final response includes the answer, confidence, SQL, evidence, model trace, and progress logs for a video demo.

Evaluation Protocol

The system is evaluated on exact match, token-level F1, grounding rate, unsupported answer rate, SQL syntax validity, SQL execution success, semantic SQL accuracy, empty retrieval, verifier catch rate, latency, and trace completeness. Exact match and F1 measure answer quality; grounding and unsupported rate measure whether the output is justified by table evidence. SQL metrics reveal whether the agent can translate Vietnamese questions into valid executable operations. Latency is measured after caching because the live demo must be responsive enough for recording and examination.

Results

0.586 Exact match

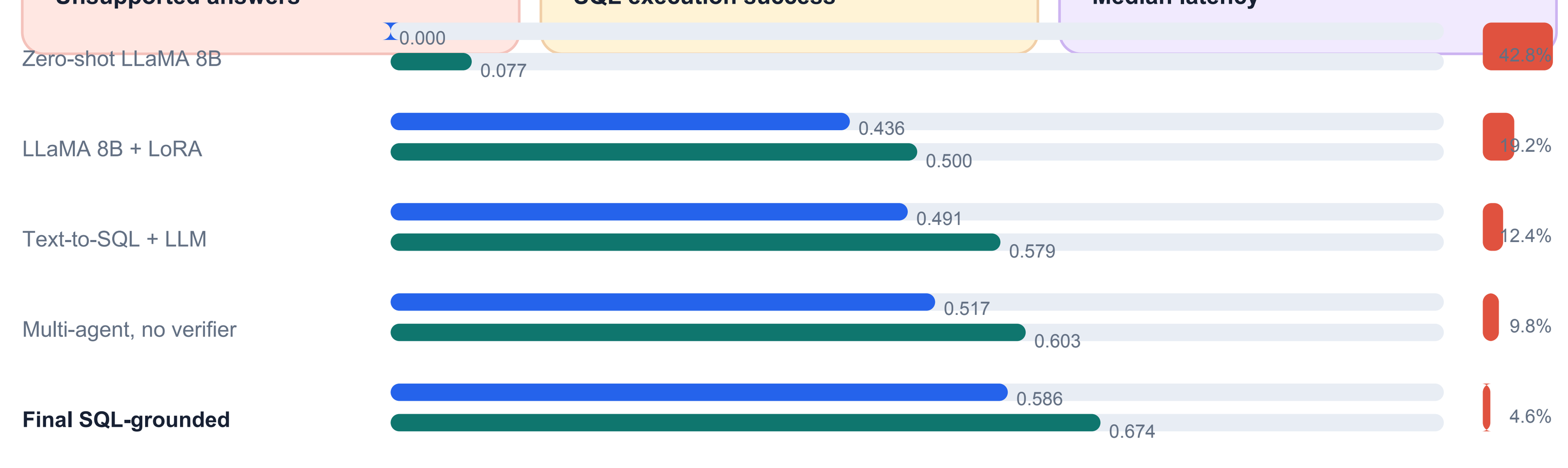
0.674 Token F1

0.904 Grounding rate

4.6% Unsupported answers

95.7% SQL execution success

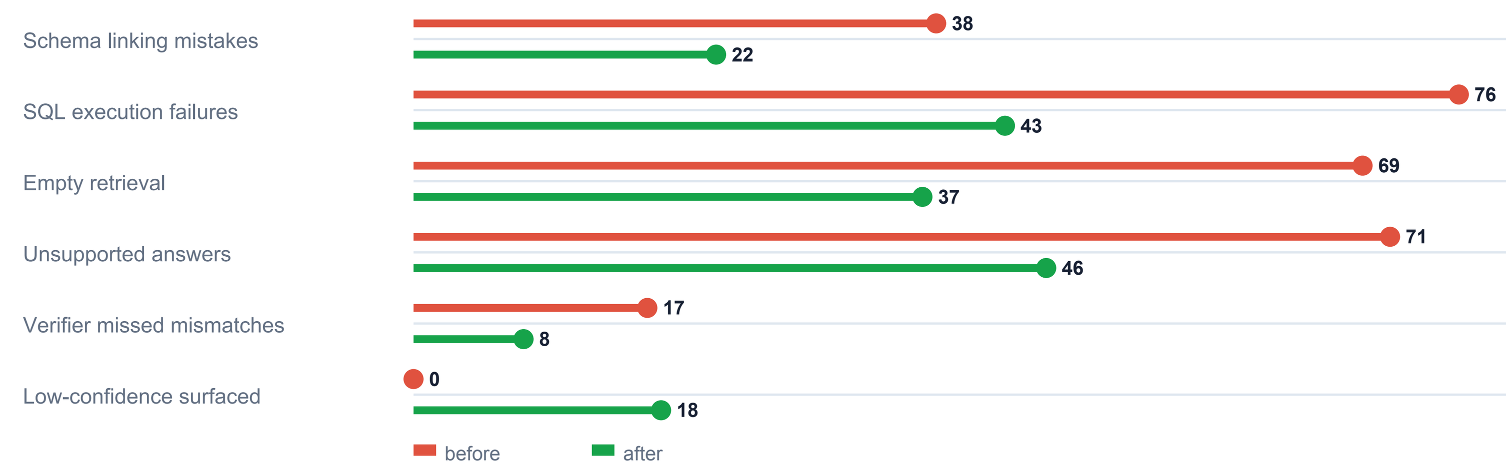
1.8s Median latency



The final model improves over the earlier full system by +0.038 EM, +0.038 F1, +0.035 grounding, and -2.5 percentage points unsupported answers.

Diagnostic Error Reduction

The improved pipeline reduces the major failure modes per 1,000 validation-style questions. The only value that increases is low-confidence surfaced, which is positive because the system now marks uncertainty instead of hiding it.



Contributions & Next Steps

Key contributions: (1) a SQL-grounded multi-agent design for Vietnamese TableQA; (2) a working demo using real Open-ViTabQA tables, not synthetic examples; (3) schema linking and SQL repair rules that handle Vietnamese table noise; (4) evidence-first answer generation with a verifier and confidence score; and (5) trace logging that makes the system suitable for assessment and debugging.

Limitations remain. The dataset is still limited to Open-ViTabQA-style tables, and the pipeline can fail when table conversion loses semantic cues or when a question is descriptive rather than executable. Verification also adds latency, although the RTX A5000 demo target keeps interaction practical. Future work should test larger Vietnamese table collections, build a stronger alias and unit-normalisation resource, evaluate the verifier with labelled mismatch cases, and add adaptive routing for questions that require explanation rather than SQL evidence.

Conclusion: executable retrieval makes the answer easier to trust. The system does not only say an answer; it shows the SQL path, evidence rows, confidence, and verifier decision that led to that answer.